Life Expectancy Prediction

Using Supervised Learning

Ridge Regression vs. Random Forest

Ruby Villalona

PROJECT INTRODUCTION & GOALS

GOAL

Predict a country's life expectancy using health and socioeconomic indicators by implementing and comparing Ridge Regression and Random Forest models.

EXPECTED OUTCOME

Identify which model better captures the complexity of real-world health data, and determine which factors most strongly drive life expectancy across countries.

DATASET DESCRIPTION

Target Variable: Life Expectancy (years)

179 Countries

2,864 Rows

21 Features

2000 – 2015 Years

index	Infant_deaths	Under_five_deaths	Adult_mortality	Alcohol_consumption	Hepatitis_B	Measles	BMI	Polio	Diphtheria	Incidents_HIV	GDP_per_capita	Population_mln
0	11.1	13.0	105.824	1.32	97	65	27.8	97	97	0.08	11006	78.53
1	2.7	3.3	57.9025	10.35	97	94	26.0	97	97	0.09	25742	46.44
2	51.5	67.9	201.0765	1.57	60	35	21.2	67	64	0.13	1076	1183.21
3	32.8	40.5	222.1965	5.68	93	74	25.3	92	93	0.79	4146	0.75
4	3.4	4.3	57.951	2.89	97	89	27.0	94	94	0.08	33995	7.91
5	9.8	11.2	95.22	4.19	88	86	26.4	89	89	0.16	9110	4.35
6	6.6	8.2	223.0	8.06	97	97	26.2	97	97	0.08	9313	144.1
7	8.7	10.1	192.969	12.23	88	99	25.9	99	99	0.08	8971	10.21
8	22.0	26.1	129.764	0.52	97	87	27.9	97	99	0.13	3708	5.22
9	15.3	17.8	217.857	7.72	97	92	26.5	96	90	0.43	2235	2.87

Key Features:

- Adult Mortality
- Under-five Deaths
- Infant Deaths
- HIV/AIDS Incidents
- GDP per Capita
- Schooling
- BMI
- Alcohol Consumption
- Immunization Rates
- Economy Status

Source: WHO Life Expectancy Fixed Dataset — Kaggle (lashagoch)

DATA PREPARATION

Load Dataset & Drop Columns

Read CSV file, sample if needed, check shape and column names
Removed Country, Region, Year

```
df = pd.read_csv('Life-Expectancy-Data-Updated.csv')
df = df.drop(columns=['Year', 'Region', 'Country'])

print(df.shape)
print(df.head())
```

```
   GDP_per_capita  Population_mln  Thinness_ten_nineteen_years  \
0           11006           78.53                          4.9
1           25742           46.44                          0.6
2            1076         1183.21                         27.1
3            4146            0.75                          5.7
4           33995            7.91                          1.2

   Thinness_five_nine_years  Schooling  Economy_status_Developed  \
0                       4.8        7.8                         0
1                       0.5        9.7                         1
2                      28.0        5.0                         0
3                       5.5        7.9                         0
4                       1.1       12.8                         1

   Economy_status_Developing  Life_expectancy
0                          1             76.5
1                          0             82.8
2                          1             65.4
3                          1             67.0
4                          0             81.7
```

Define X and y

X = 17 features, y = Life Expectancy target variable

```
X = df.drop(columns=['Life_expectancy'])
y = df['Life_expectancy']

print(X.shape)
print(y.shape)
```

```
(2864, 17)
(2864,)
```

```
(2864, 18)
   Infant_deaths  Under_five_deaths  Adult_mortality  Alcohol_consumption  \
0           11.1               13.0         105.8240                 1.32
1            2.7                3.3          57.9025                10.35
2           51.5               67.9         201.0765                 1.57
3           32.8               40.5         222.1965                 5.68
4            3.4                4.3          57.9510                 2.89

   Hepatitis_B  Measles   BMI  Polio  Diphtheria  Incidents_HIV  \
0           97       65  27.8     97          97           0.08
1           97       94  26.0     97          97           0.09
2           60       35  21.2     67          64           0.13
3           93       74  25.3     92          93           0.79
4           97       89  27.0     94          94           0.08
```

PREPROCESSING

Train/Test Split

80/20 split using train_test_split (random_state=42)

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
```

```
(2291, 17)
(573, 17)
(2291,)
(573,)
```

StandardScaler: fit_transform on X_train, transform only on X_test

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

RIDGE REGRESSION

A linear model with L2 regularization. It assumes a straight-line relationship between features and life expectancy, penalizing large coefficients to prevent overfitting.

```
param_grid_ridge = {'alpha': [0.01, 0.1, 1, 10, 100]}

ridge = GridSearchCV(Ridge(), param_grid_ridge, cv=5, scoring='r2')
ridge.fit(X_train, y_train)

print("Best Alpha:", ridge.best_params_)
print("Best R2 Score:", ridge.best_score_)


y_pred_ridge = ridge.predict(X_test)

print("Test R2 Score:", r2_score(y_test, y_pred_ridge))
print("Test MSE:", mean_squared_error(y_test, y_pred_ridge))
print("Test MAE:", mean_absolute_error(y_test, y_pred_ridge))
```

Hyperparameter Tuning (GridSearchCV, cv=5)

Parameter tested: alpha = [0.01, 0.1, 1, 10, 100]
Best alpha found: 1
Best CV R^2: 0.979

0.977
R^2 Score

1.853
MSE

1.09 yrs
MAE

RANDOM FOREST

An ensemble model that builds many decision trees and averages their predictions. Unlike Ridge, it captures non-linear relationships — the same feature can have different effects depending on other factors.

Hyperparameter Tuning (RandomizedSearchCV, cv=5, n_iter=10)

n_estimators: [50, 100, 200] → Best: 100
max_depth: [None, 3, 5, 10] → Best: 10
min_samples_split: [2, 5, 10] → Best: 5
Best CV R^2: 0.9946

```python
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 3, 5, 10],
    'min_samples_split': [2, 5, 10],
}

grid_search = RandomizedSearchCV(
    RandomForestRegressor(random_state=42, n_jobs=-1),
    param_grid,
    n_iter=10,
    cv=5,
    scoring='r2',
    random_state=42,
    n_jobs=-1
)
grid_search.fit(X_train, y_train)

print("Best Parameters:", grid_search.best_params_)
print("Best R2 Score:", grid_search.best_score_)
```

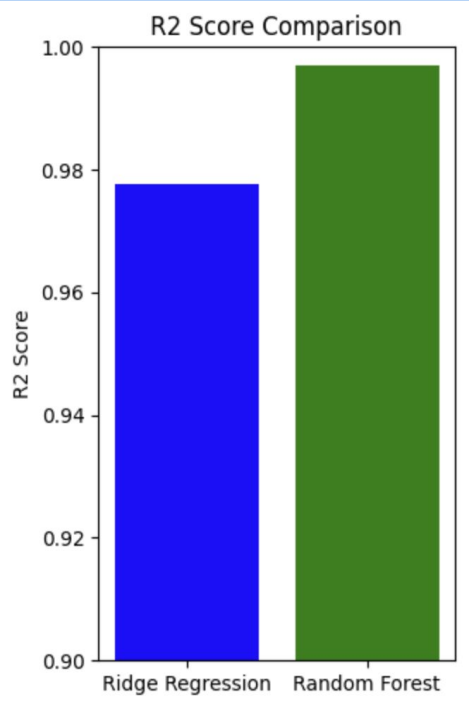
```python
best_rf = grid_search.best_estimator_

y_pred_rf_tuned = best_rf.predict(X_test)

print("R2 Score:", r2_score(y_test, y_pred_rf_tuned))
print("MSE:", mean_squared_error(y_test, y_pred_rf_tuned))
print("MAE:", mean_absolute_error(y_test, y_pred_rf_tuned))
```

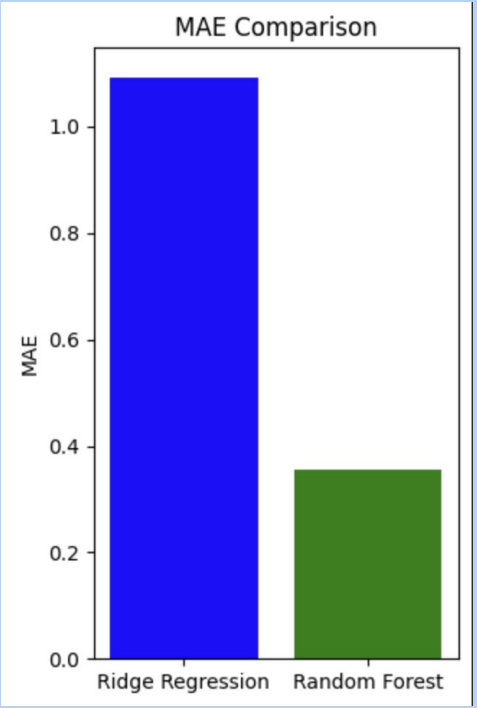
0.996
R^2 Score

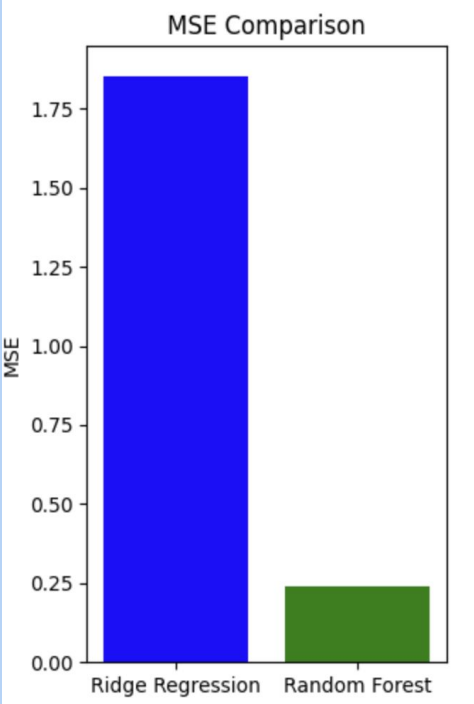
0.333
MSE

0.444 yrs
MAE

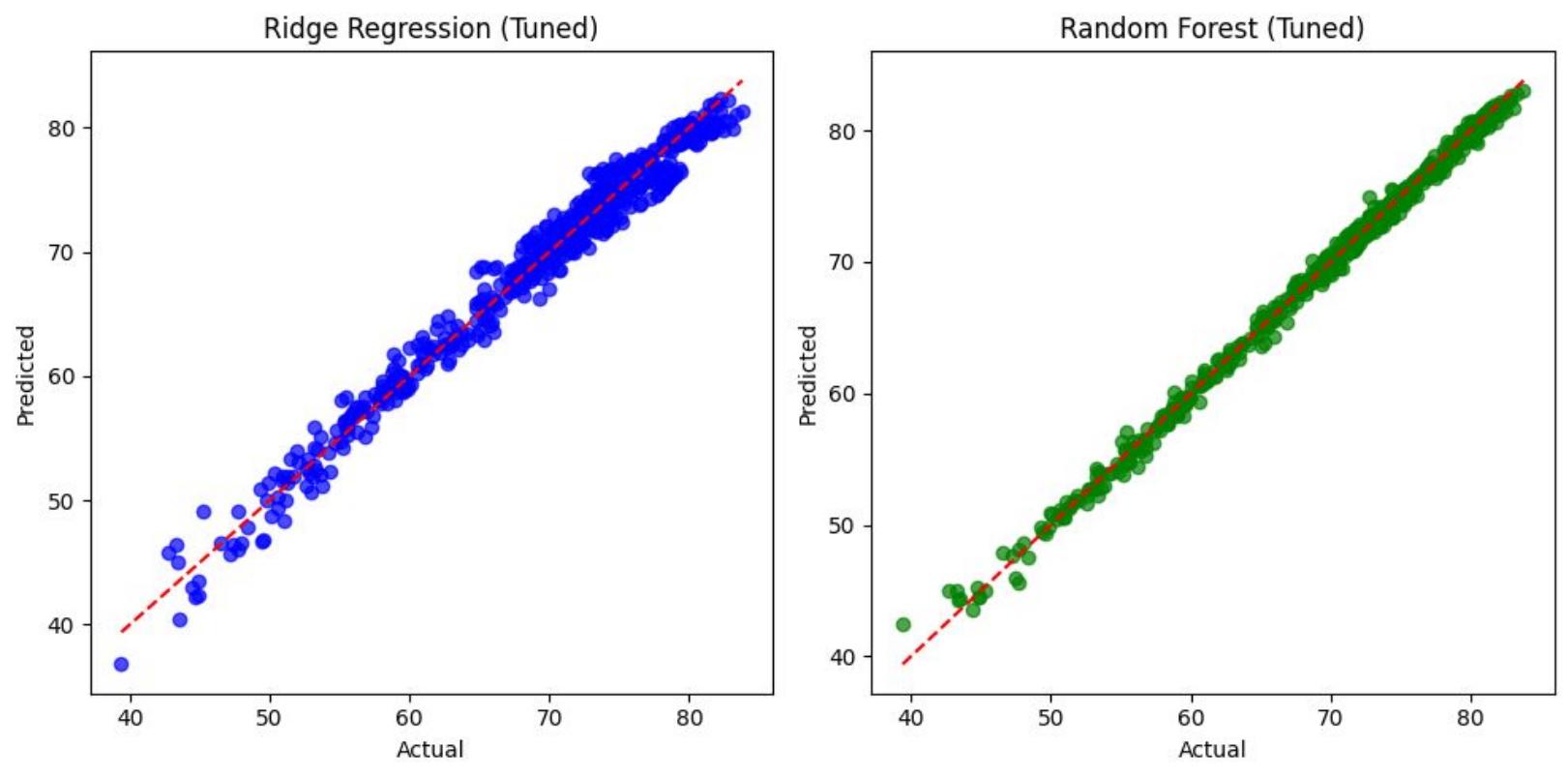
PERFORMANCE VISUALIZATION





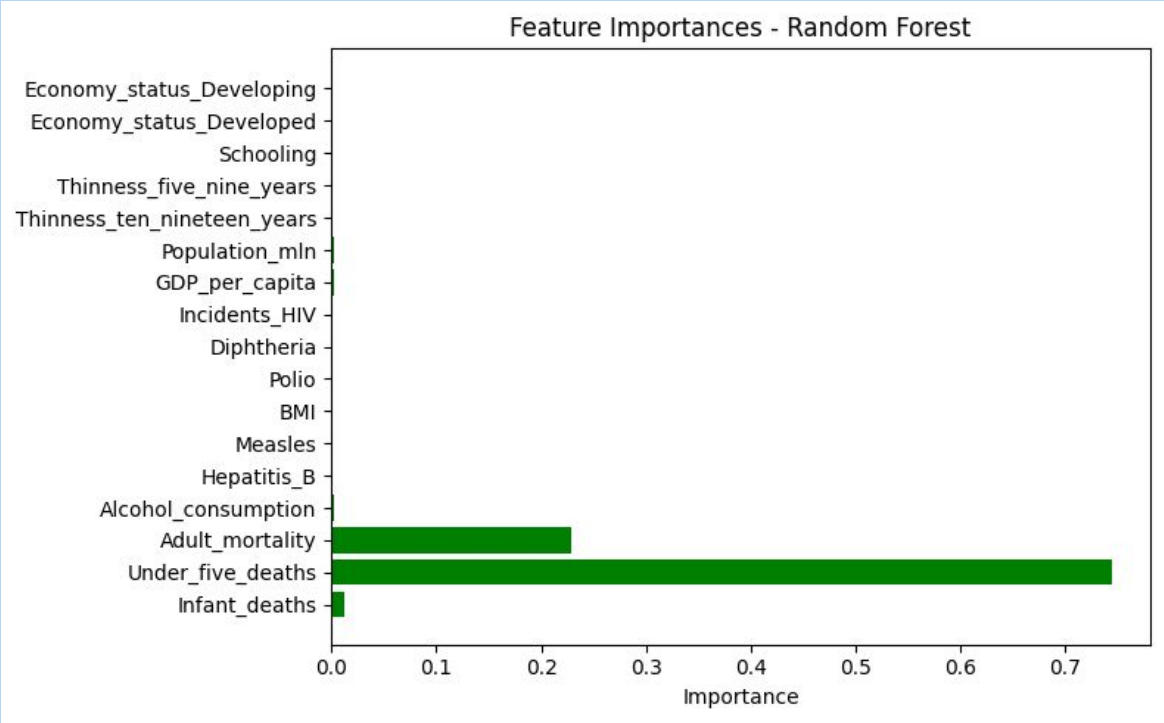


SCATTER PLOTS



Ridge (left): dots are more scattered — larger prediction errors. *Random Forest (right): dots tightly follow the perfect prediction line — more accurate.*

FEATURE IMPORTANCE — RANDOM FOREST



Key Findings

- Under-five deaths is the strongest predictor by far (0.75 importance)
- Adult mortality is second (0.25 importance)
- GDP, schooling, and HIV contribute very little in comparison
- Countries that protect their youngest children have dramatically higher life expectancy

CONCLUSIONS

Random Forest outperforms Ridge on every metric

R^2 of 0.996 vs 0.977 and MAE of 0.445 vs 1.09 years — Random Forest is significantly more accurate.

Non-linear relationships drive the gap

Ridge assumes straight-line relationships; Random Forest captures complex interactions between health indicators.

Child mortality is the #1 predictor of life expectancy

Under-five deaths scored 0.75 importance — far higher than GDP, education, or HIV rates.

Policy implication

Reducing child mortality should be the top priority for improving a country's overall population health.